# Authorization Testing Lab

10 integration tests / Spring TestContext + MockMvc / OAuth 2.0 Resource Server

Your starting point is **Document Vault**, a working resource server with authorization rules already implemented. Write the **10 tests** in this worksheet. Each task asks for **one @Test method with one HTTP request** and introduces a different authorization concept.

## Your workspace

Complete **AuthorizationRulesTest** in src/test/java. It extends **AbstractAuthorizationTest**, which loads the application using @SpringBootTest and @AutoConfigureMockMvc with security filters enabled. The base class provides the protected fields **mockMvc**, **documentRepository**, and **jwtDecoder**.

The controllers, services, repository, and security configuration are real application components. Only JwtDecoder is replaced using @MockitoBean. Keep src/main unchanged and leave the security filters enabled. Do not use @WithMockUser, standaloneSetup, or service mocks.

## Preparing test identities

For tests 1 and 2, do not attach authentication. For tests 3-10, use **jwt()** from Spring Security Test, set **sub** explicitly, and supply only the authorities listed in the task. For test 9, also set the tenant claim. Use **SimpleGrantedAuthority** for the exact authority values specified.

Scope authorities use the **SCOPE_** prefix; role authorities use **ROLE_**. The jwt() request post-processor supplies test authentication directly. It does not execute the application's JWT decoder or converter, so you do not need signed tokens or decode() stubs.

## Initial data

| id | title | owner | tenant | archived |
|---|---|---|---|---|
| d-101 | Spring notes | ana | alpha | false |
| d-102 | Test strategy | bogdan | beta | false |

Before each test, the base class calls documentRepository.reset(). Document has exactly the fields shown above. findById(id) returns Optional<Document>. The first document created after a reset has id=d-103. There are no role or scope hierarchies and no global ADMIN bypass.

## Running your tests

Run **mvn test**. Use Java 21, Maven 3.9+, and Spring Boot 4.1.2. The API is stateless and CSRF is disabled; do not add csrf() to requests. Use descriptive test names and **do not add code comments**. Estimated time: 60-90 minutes.

# Tests 1-4

Public access, authentication failures, authorization failures, and an authorized write

## 1 Public access

**Rule:** GET /api/public/info is public.

**Your request:** Send GET /api/public/info without a JWT or an Authorization header.

**Assert:** HTTP 200 and the JSON field name equal to document-vault.

**Learning goal:** Recognize how permitAll allows anonymous requests.

## 2 Missing authentication

**Rule:** GET /api/me requires an authenticated user.

**Your request:** Send GET /api/me without a JWT.

**Assert:** HTTP 401 and a WWW-Authenticate header whose value starts with Bearer.

**Learning goal:** Distinguish an unauthenticated request from an authorization failure.

## 3 Insufficient authority

**Rule:** DELETE /api/documents/{id} requires SCOPE_documents.delete.

**Your request:** Send DELETE /api/documents/d-101 with sub=ana and only the authority SCOPE_documents.read.

**Assert:** HTTP 403.

**Learning goal:** An authenticated user can still lack permission for an operation.

## 4 An authorized write

**Rule:** POST /api/documents requires SCOPE_documents.write.

**Your request:** With sub=ana and only SCOPE_documents.write, send POST /api/documents with Content-Type application/json and body {"title":"Security notes","tenant":"alpha"}.

**Assert:** HTTP 201, Location=/api/documents/d-103, and JSON containing id=d-103, title=Security notes, owner=ana, tenant=alpha, archived=false.

**Learning goal:** Supply an explicit scope and verify that an authorized operation succeeds.

# Tests 5-8

Roles, AND/OR expressions, and the authenticated identity

## 5 The administrator role

**Rule:** GET /api/admin/audit requires the ADMIN role.

**Your request:** Send GET /api/admin/audit with sub=operator and only the authority ROLE_ADMIN. Do not add any scopes.

**Assert:** HTTP 200 and the JSON field eventCount equal to 2.

**Learning goal:** Understand that hasRole("ADMIN") checks for the authority ROLE_ADMIN.

---

## 6 An AND policy

**Rule:** GET /api/reports/export requires SCOPE_reports.read AND SCOPE_reports.export.

**Your request:** Send GET /api/reports/export with sub=ana and only SCOPE_reports.read.

**Assert:** HTTP 403 because SCOPE_reports.export is missing.

**Learning goal:** A conjunction requires every condition to be satisfied.

---

## 7 An OR policy

**Rule:** GET /api/reports/summary requires SCOPE_reports.read OR ROLE_AUDITOR.

**Your request:** Send GET /api/reports/summary with sub=auditor and only ROLE_AUDITOR, without SCOPE_reports.read.

**Assert:** HTTP 200 and the JSON field documentCount equal to 2.

**Learning goal:** A disjunction grants access when either condition is satisfied.

---

## 8 The user in the URL

**Rule:** GET /api/users/{userId}/documents requires SCOPE_documents.read AND userId equal to sub. ADMIN has no exception.

**Your request:** Send GET /api/users/bogdan/documents with sub=ana and only SCOPE_documents.read.

**Assert:** HTTP 403: ana cannot request bogdan's documents.

**Learning goal:** Compare the authenticated identity with a path variable.

---

# Tests 9-10

JWT claims and service method authorization

## 9 The tenant claim

**Rule:** GET /api/tenants/{tenantId}/documents requires SCOPE_documents.read AND the tenant claim equal to tenantId. ADMIN has no exception.

**Your request:** Send GET /api/tenants/beta/documents with sub=ana, the claim tenant=alpha, and only SCOPE_documents.read.

**Assert:** HTTP 403 because the JWT tenant does not match the tenant in the URL.

**Learning goal:** Use a JWT claim as part of an authorization decision.

---

## 10 Method security and side effects

**Rule:** PATCH /api/documents/{id}/archive requires only authentication at the HTTP layer. The service method requires SCOPE_documents.write AND the document owner equal to sub.

**Your request:** Send PATCH /api/documents/d-101/archive without a request body, with sub=bogdan and only SCOPE_documents.write.

**Assert:** HTTP 403. Read d-101 from documentRepository and assert that archived is still false. Keep the service real; do not replace it with a mock.

**Learning goal:** Exercise @PreAuthorize on a real Spring service and check that denied operations leave state unchanged.

---

## What to submit

Submit AuthorizationRulesTest with **10 @Test methods** and the result of a successful mvn test run. Each method should implement the single request and assertions in its task. Keep the supplied infrastructure unchanged. No additional test cases are required.

## Useful APIs

Use get(), post(), delete(), patch(), status(), jsonPath(), and header(). For test 10, inspect documentRepository.findById("d-101").orElseThrow().archived() directly instead of making another HTTP request. For 401/403 responses, do not assert the exact error response body.

JWT authentication in MockMvc tests

HTTP request authorization

Method security